

Workshop Job Card MVP - Legacy Data Mapping Specification

This document defines how the existing legacy CSV exports map into the new workshop application database.

It is based on:

- the approved MVP scope
- the revised wireframes
- the database schema document
- your confirmed migration rules

Confirmed migration decisions:

- Default imported customer type = **retail**
- Old visible job card number should be preserved in a separate **legacy_jobcard_number** field
- If an old job card has incomplete customer data, unresolved links should go to **Admin review**, but the system should show a **recommended link** when vehicle match is strong

1. Purpose of this document

This mapping specification defines:

- which source file maps to which target table
- how source columns map to target fields
- normalization rules
- duplicate and merge rules
- unresolved data handling
- read-only import behavior
- what is imported now vs later

This document is intended for:

- developer implementation of the import engine
- admin import profile design
- import preview and duplicate review workflow

2. Source files in scope

The uploaded legacy files are:

- `Partners.csv`
- `Auto object table.csv`
- `Job cards.csv`
- `Problems.csv`
- `Items.csv`
- `Items 1.csv`
- `Deadlines.csv`

3. High-level file-to-table mapping

Source file	Main target tables	Import priority	Notes
Partners.csv	customers	High	customer master import
Auto object table.csv	vehicles, vehicle_ownership_history, reminder_events	High	vehicle master import
Job cards.csv	jobcards, jobcard_snapshots	High	historical job cards
Problems.csv	jobcard_complaints, complaint_icon_master seed review	Medium	complaints history/presets
Items.csv	parts_master	High	product/parts catalog
Items 1.csv	labour_master	High	services/labour catalog
Deadlines.csv	reminder_events	Medium	reminders where data is clear

4. Global import rules

4.1 Import mode behavior

- All first-time imports should begin as ****dry run****.
- No import goes live without preview.
- Exact matches may be auto-classified.
- Likely matches must go to review if confidence is below rule threshold.
- Unknown columns must be stored in ``import_batch_rows.unknown_fields_json``.

4.2 Legacy record behavior

- Imported historical job cards must be marked:
 - ``legacy_import_flag = true``
 - ``read_only_flag = true``
- Imported masters remain editable unless otherwise locked.

4.3 Source traceability

Every imported row should preserve:

- ``source_system``
- ``source_record_id``
- import batch id
- raw source row JSON snapshot

4.4 Date handling

Legacy files appear to use ``DD-MM-YYYY`` format.

Importer must parse day-first dates explicitly.

4.5 Money handling

Legacy amounts may contain commas, such as ``24,110.00``.

Importer must strip thousand separators before numeric conversion.

4.6 Registration number normalization

Source registration values must be normalized by:

- uppercasing
- trimming whitespace
- removing spaces and dashes for normalized key

Example:

- `HR 26EN 8059`
- `HR26EN8059`

Both become:

- raw: original input preserved
- normalized: `HR26EN8059`

4.7 Customer default type

If customer type is not present in legacy data:

- default to `retail`

5. Partners.csv -> customers

5.1 Purpose

This file is the primary source for customer master records.

5.2 Expected source columns observed

Examples include:

- Id
- Data type
- Name
- Address
- Settlement
- State
- ZIP
- Country
- Vat number
- Phone number
- E-mail
- Bank account
- Mail list
- Deadline
- Mobile Number
- Priority
- Driver Name
- Notes
- Created
- Creator

5.3 Target table

- `customers`

5.4 Column mapping

Source column	Target field	Rule
Id	source_record_id	store as text
Name	display_name	required for customer creation
Address	address_line_1	full text for MVP
State	state	map directly if clean
ZIP	postal_code	keep as text
Country	country	default IN if blank
Vat number	gst_number	map if valid
Phone number	primary_mobile	normalize phone
Mobile Numer	alternate_mobile or primary_mobile	fallback use if primary blank
E-mail	email	lowercase trim
Notes	notes	append as text
Created	created_at	source reference may be stored in source metadata only

5.5 Default values

- `customer\_type` = retail`
- `is\_active` = true`
- `source\_system` = 'legacy_csv_partners`

5.6 Duplicate rules

Strong customer match priority:

1. GST number exact
2. primary mobile exact
3. alternate mobile exact
4. email exact
5. normalized name + address similarity

5.7 Merge rules

- if exact mobile/email/GST match exists -> candidate update
- if likely name/address match only -> review queue
- notes should append, not overwrite
- blank fields in target may be filled from source

5.8 Data quality concerns

- `Mobile Numer` appears misspelled in source header and may contain alternate phone
- names may include business shorthand
- addresses are often unstructured and may remain in one field initially

6. Auto object table.csv -> vehicles, ownership, reminders

6.1 Purpose

Primary source for vehicle master records.

6.2 Expected source columns observed

Examples include:

- Id
- Data type
- LPN
- Manufacturer
- Model
- Engine type
- Fuel type
- VIN
- Engine number
- Manufacture year
- Color
- Battery Details
- Next service
- Next P.U.C
- Next oil change
- Next oil change dist.
- Next timing belt change
- Insurer
- Customer id.
- Customer Address
- Phone number
- E-mail
- Bank account
- Mail list
- Deadline
- Mobile Numer
- Priority
- Driver Name
- Notes
- Created
- Creator

6.3 Target tables

- `vehicles`
- `vehicle\_ownership\_history`
- `reminder\_events`

6.4 Column mapping to vehicles

Source column	Target field	Rule
Id	source_record_id	store as text
LPN	registration_number_raw	preserve original
LPN	registration_number_normalized	uppercase/remove spaces-dashes
VIN	vin	direct map
Engine number	engine_number	direct map
Manufacturer	manufacturer	direct map
Model	model	direct map
Engine type	variant or notes	use cautiously; may be engine descriptor
Fuel type	fuel_type	direct map

```
| Manufacture year | manufacture_year | integer parse |
| Color | color | direct map |
| Battery Details | battery_details | direct map |
| Next service | next_service_date | parse date |
| Next P.U.C | next_puc_date | parse date |
| Next oil change | next_oil_change_date | parse date |
| Next oil change dist. | next_oil_change_distance | integer parse |
| Next timing belt change | next_timing_belt_change_date | parse date |
| Insurer | insurer_name | direct map |
| Notes | notes | direct map |
| Customer id. | current_customer_id | resolve via imported customers
source_record_id |
```

6.5 Ownership history rule

On import, create one ownership history row if customer link is resolved:

- `vehicle\_id`
- resolved `customer\_id`
- `from\_date = null` if unknown
- `to\_date = null`
- mark as current ownership

6.6 Reminder mapping

If date exists, create reminder rows:

- `Next service` -> `next\_service`
- `Next oil change` -> `next\_oil\_change`
- `Next P.U.C` -> `next\_puc`
- insurance expiry is only importable if clearly present later

6.7 Duplicate rules

Strong vehicle match priority:

1. normalized registration number exact
2. VIN exact
3. engine number exact
4. normalized registration partial + make/model similarity -> review

6.8 Merge rules

- registration number is primary identity
- fill missing VIN, engine number, make, model, color, reminder dates from source where target blank
- if conflicting active registration exists, send to review queue

6.9 Data quality concerns

- some LPN values may not be clean registration numbers
- some rows may include partial customer data in vehicle file
- phone/email may support customer match when customer id is weak

7. Job cards.csv -> jobcards, jobcard_snapshots

7.1 Purpose

Import historical service transactions.

7.2 Expected source columns observed

Examples include:

- Id
- Warn
- Data type
- State
- Invoice
- Auto id
- LPN
- Manufacturer
- Model
- Engine type
- Fuel type
- VIN
- Engine number
- Manufacture year
- Color
- Battery Details
- Next service
- Next P.U.C
- Next oil change
- Next oil change dist.
- Next timing belt change
- Insurer
- Auto notes
- Customer id.
- Customer Address
- Created
- Deadline
- Completed
- Closed
- Paid
- Net sum
- Tax sum
- Gross sum
- Deposit
- Payable
- Odometer mileage

7.3 Target tables

- `jobcards`
- `jobcard\_snapshots`

7.4 Key import behavior

- Historical job cards are imported as **legacy read-only** records.
- Visible old number must be stored in **legacy_jobcard_number**.
- New app-generated numbering applies only to future job cards.

7.5 Required schema note

The `jobcards` table should include an additional field:

- `legacy\_jobcard\_number` (text, nullable)

7.6 Column mapping to jobcards

Source column	Target field	Rule
Id	legacy_jobcard_number	preserve visible old number
Id	source_record_id	source trace
State	status	map legacy status carefully
Created	date_in	parse date
Deadline	expected_delivery_at	parse date if valid
Closed	closed_at or payment/closure indicator	source appears mixed; use carefully
Paid	payment_status	map yes/no if available
Net sum	subtotal_amount	numeric parse
Tax sum	tax_amount	numeric parse
Gross sum	total_amount	numeric parse
Deposit	optional financial note or future deposit field	if schema extended later
Payable	total_amount fallback or balance note	use carefully
Odometer mileage	intake_odometer	integer parse
Auto id	vehicle source link	resolve through source vehicle id if possible
Customer id.	customer source link	resolve if available
LPN	vehicle matching backup	normalize and match vehicle
Customer Address	snapshot customer_address	direct map
Manufacturer	snapshot vehicle_make	direct map
Model	snapshot vehicle_model	direct map
Color	snapshot vehicle_color	direct map
VIN	snapshot vehicle_vin	direct map

7.7 Status mapping proposal

Legacy `State` values should map like this unless source confirms more values:

- Opened -> `open`
- Closed -> `closed`
- unknown/other -> review or fallback `legacy\_imported\_read\_only`

7.8 Linking logic

Priority for linking imported job card:

1. source Auto id -> imported vehicle source_record_id
2. normalized LPN -> vehicle registration match
3. source Customer id -> imported customer source_record_id
4. if customer unresolved and vehicle resolved:
 - leave unresolved for admin review
 - show recommended customer link if current vehicle owner is strong match

7.9 Snapshot creation

Create one `jobcard\_snapshots` row per imported job card using best available fields

from source row.

7.10 Default values

- `legacy\_import\_flag = true`
- `read\_only\_flag = true`
- `source\_system = 'legacy\_csv\_jobcards'`

7.11 Data quality concerns

- some source rows may be incomplete
- state/closed/paid semantics may need exact source validation during pilot import
- line-item parts/labour are not present in this file and cannot be fully reconstructed from this file alone

8. Problems.csv -> complaints history and preset seeding

8.1 Purpose

This file contains issue/problem text that can support:

- historical complaint import where linkable
- complaint preset seeding for icon/text suggestions

8.2 Expected source columns observed

Examples include:

- Id
- List id
- Data type
- Type
- Solved
- Name
- Created
- Creator

8.3 Target tables

Primary import use:

- `jobcard\_complaints` when link to job/list is resolvable

Secondary admin review use:

- seed recommendations for `complaint\_icon\_master`

8.4 Mapping logic

Source column	Target field	Rule
Id	source_record_id	trace only
List id	possible jobcard/source link	resolve if relationship can be inferred
Name	customer_complaint_text	main text payload
Solved	note or future solved state	not core MVP
Created	created_at source reference	optional

8.5 Import rule

Because linkage may be weak, use a two-mode approach:

- if `List id` can reliably map to a legacy job card -> import to `jobcard\_complaints`
- otherwise -> store as preset candidate / raw import review set

8.6 Complaint preset seeding

Common repeated phrases can be reviewed by admin and converted into:

- complaint icons
- complaint text suggestions
- service templates

8.7 Data quality concerns

- complaint rows may not map 1:1 to current job cards without list relationship understanding
- some entries are very short or workshop shorthand

9. Items.csv -> parts_master

9.1 Purpose

Primary source for parts/products catalog.

9.2 Expected source columns observed

Examples include:

- Id
- Type
- Intermed.
- Name
- Item code
- Part number
- Factory num.
- Barcode
- Class. num.
- Stock
- Unit
- Cost tax r.
- Net cost
- Cost tax
- Gross cost
- Net cost sum
- Cost tax sum
- Gross cost sum
- Disc.
- Tax r.
- Net list price
- List price tax
- Gross list price
- Net price

- Tax
- Gross price
- Net sum
- Tax sum
- Gross s.
- Profit
- Seller
- Created

9.3 Target table

- `parts\_master`

9.4 Column mapping

Source column	Target field	Rule
Id	source_record_id	trace
Name	part_name	required
Item code	item_code	direct map
Part number	part_number	direct map
Barcode	barcode	direct map
Unit	unit	direct map
Tax r.	default_tax_rate	numeric percent
Net price / Gross price / Net list price	default_selling_price	choose best business field
Stock	stock_quantity	numeric parse
Seller	brand or seller note	map cautiously
Created	source created	optional

9.5 Pricing rule

Per your decision, store:

- selling price
- not required to store cost price in MVP schema

9.6 Duplicate rules

Priority:

1. item_code exact
2. part_number exact
3. barcode exact
4. exact normalized name
5. fuzzy name similarity -> review

9.7 Merge rules

- preserve active part master
- fill missing codes/barcodes
- update selling price only if import mode allows

9.8 Data quality concerns

- source may include products not actively used anymore
- some rows may reflect transactional history rather than clean master data

- negative/zero pricing rows need preview review

10. Items 1.csv -> labour_master

10.1 Purpose

Primary source for labour/service catalog.

10.2 Expected source columns observed

Examples include:

- Id
- Type
- Intermed.
- Name
- Item code
- Part number
- Class. num.
- Unit
- Disc.
- Tax r.
- Net list price
- List price tax
- Gross list price
- Net price
- Tax
- Gross price
- Net sum
- Tax sum
- Gross s.
- Seller
- Created

10.3 Target table

- `labour\_master`

10.4 Column mapping

Source column	Target field	Rule
Id	source_record_id	trace
Name	labour_name	required
Item code	labour_code	direct map when present
Unit	unit_type	direct map
Tax r.	default_tax_rate	numeric percent
Net price / Gross price / Net list price	default_selling_price	choose best field
Created	source created	optional

10.5 Pricing rule

Per your decision, store:

- selling price
- no internal cost required for MVP import

10.6 Duplicate rules

Priority:

1. labour_code exact
2. exact normalized labour_name
3. fuzzy title similarity -> review

10.7 Data quality concerns

- some service names may be detailed free-text and need later cleanup
- some entries may be one-off services, not standard labour catalog items

11. Deadlines.csv -> reminder_events

11.1 Purpose

Secondary reminder import source.

11.2 Expected source columns observed

Examples include:

- Identifier
- Type
- Denomination
- Date
- Left
- Partner
- Address

11.3 Target table

- `reminder\_events`

11.4 Mapping logic

Source column	Target field	Rule	
---	---	---	
Identifier	source_record_id	/ source vehicle id	resolve to vehicle where possible
Denomination	reminder_type		map next service / next oil change
Date	due_date		parse
Partner	customer hint		optional support for review
Address	support note		optional

11.5 Import rule

- import only when identifier can be linked strongly to an imported vehicle
- unresolved reminders should go to review queue, not silent import

11.6 Reminder type mapping

- `Next service` -> `next\_service`
- `Next oil change` -> `next\_oil\_change`
- future use if more categories exist

12. Recommended additional schema adjustments

To support the migration cleanly, these extra fields are recommended:

12.1 jobcards

Add:

- `legacy\_jobcard\_number` text nullable

12.2 customers, vehicles, parts_master, labour_master

Already recommended and confirmed:

- `source\_system`
- `source\_record\_id`

12.3 import_batch_rows

Keep:

- `unknown\_fields\_json`
- `raw\_row\_json`

13. Unresolved link handling rules

13.1 Customer unresolved but vehicle resolved

Behavior:

- do not auto-link final customer
- create admin review item
- show recommended customer = current owner of matched vehicle when confidence is strong

13.2 Vehicle unresolved but customer resolved

Behavior:

- create admin review item
- if registration is blank or invalid, do not force import to live vehicle master without review

13.3 Both unresolved

Behavior:

- import row remains pending review
- no live job card created until reviewed, unless import mode explicitly allows partial legacy staging

14. Suggested import order

For first migration, import in this order:

1. `Partners.csv` -> customers
2. `Auto object table.csv` -> vehicles + ownership + reminders
3. `Items.csv` -> parts master
4. `Items 1.csv` -> labour master
5. `Job cards.csv` -> legacy job cards + snapshots
6. `Problems.csv` -> complaints/presets review
7. `Deadlines.csv` -> remaining reminders

This order gives the job card importer the best chance to resolve customer and vehicle references.

15. Admin import review design implications

The import UI should support these review states:

- exact match
- likely match
- unknown field
- ignored field
- conflict field
- unresolved link
- recommended link available

For unresolved job cards, the review screen should show:

- legacy job number
- registration number from source
- matched vehicle recommendation if any
- matched customer recommendation if any
- approve / keep unresolved / create new

16. MVP import limitations to acknowledge

These source files do ****not**** appear to provide a full clean relational export of:

- exact line-level parts per historical job card
- exact line-level labour per historical job card
- clean one-to-one complaint-to-job mapping in all cases
- complete structured billing breakdown for every scenario

Therefore MVP migration should focus on:

- strong vehicle history
- customer history
- legacy job headers
- complaint text where linkable
- master catalogs for parts and labour

Detailed historical line reconstruction may require later review or additional exports.

17. Recommended next technical document

After this mapping spec, the next document should be:

Developer-ready technical schema

That should include:

- SQL or Prisma models
- enums
- foreign keys
- unique constraints
- indexes
- import-specific staging tables if needed

After that:

- API specification
- sprint plan
- build start